Smalltalk w projekcie GoSmall

Jakie narzędzia wybrać?

1. GNU Smalltalk – można zainstalować w IDE ogólnego przeznaczenia, np. w VS Code
2. IDE dedykowane smalltalkowi
    a. Cincom VisualWorks (free dla studentów)
    b. Squeak (free)
3. Interpretery on-line:
    a. https://www.jdoodle.com/execute-smalltalk-online/

Pierwsza próba w GNU Smalltalk

Zapisz poniższy fragment w pliku `proba.st` i uruchom poleceniem `gst proba.st`. Pionowe kreski otaczają nazwy zmiennych lokalnych, `:=` przypisuje wartość, a kropka rozdziela instrukcje (co można rozumieć jako koniec zdania).

```
| minuty |
minuty := 12 + 8.
Transcript show: minuty printString; cr.
```

W konsoli pojawi się `20`. `printString` zamienia liczbę na tekst. Średnik wysyła kolejny komunikat do tego samego odbiorcy, więc zarówno `show:`, jak i `cr` trafiają do `Transcript`. `cr` kończy wiersz i przechodzi do nowej linii.

Komunikaty

Jak zapisać podróż złożoną ze spaceru i przejazdu autobusem? W Smalltalku obiekty wymieniają między sobą komunikaty. Mogą wystąpić trzy rodzaje komunikatów:

```
pieszo czas                 "unary: bez argumentu"
pieszo + autobus            "binary: jeden argument – inny obiekt"
pieszo ustawCzas: 15        "keyword: słowo z dwukropkiem"
```

Pierwszy obiekt to **odbiorca**. W ostatnim wierszu `ustawCzas:` jest nazwą komunikatu, a `15` jego argumentem (innym obiektem). Nazwa może mieć kilka części, np. w komunikacie `od: do:` każdemu dwukropkowi odpowiada jeden argument.

W jakiej kolejności czytać komunikaty

Najpierw wykonują się komunikaty unary, potem binary, na końcu keyword. Wszystkie komunikaty binarne mają ten sam priorytet i wykonują się od lewej. Nawiasy pozwalają zmienić kolejność.

```
2 + 3 * 4       "20"
2 + (3 * 4)     "14"
```

Klasa odcinka i jego stan

Dwa odcinki mogą mieć różny czas przejazdu. Dlatego minuty przechowujemy w zmiennej instancji. Każdy odcinek ma własną wartość tej zmiennej.

```
Object subclass: OdcinekPodrozy [
    | minuty |

    OdcinekPodrozy class >> new [
       |op|
        op := super new.
        op ustawCzas: 0.
        ^op
    ]

    ustawCzas: liczba [
        minuty := liczba.
    ]

    czas [ ^minuty ]
]
```

`subclass:` tworzy podklasę `Object`. Metoda po stronie klasy (`class >> new`) odpowiada na komunikat wysłany do klasy. `super new` tworzy instancję (jest konstruktorem), a `ustawCzas:` nadaje jej początkowy czas równy zero.

Komunikaty `czas` i `ustawCzas:` działają po stronie instancji. `self` oznacza bieżącego odbiorcę, a `^` kończy metodę i zwraca wskazany obiekt. Dzięki `^self` wynikiem `ustawCzas:` jest obiekt typu `OdcinekPodrozy`.

Sprawdź niezależność instancji

```
| pieszo autobus |
pieszo := OdcinekPodrozy new.
autobus := OdcinekPodrozy new ustawCzas: 18.
Transcript show: pieszo czas printString; cr.    "0"
Transcript show: autobus czas printString; cr.  "18"
```

Umieść definicję klasy przed kodem próby w tym samym pliku. Późniejsze przykłady z nową deklaracją zmiennych lokalnych uruchamiaj jako osobne próby po załadowaniu odpowiednich klas. W cudzysłowie umieszcza się komentarz, a łańcuchy znaków w apostrofach.

Dziedziczenie i wybór metody

Odcinek autobusowy może uwzględniać czas potrzebny na wejście do pojazdu. W tym ćwiczeniu doliczamy umowne dwie minuty. Podklasa dziedziczy `ustawCzas:`, ale podaje czas według własnej reguły.

```
OdcinekPodrozy subclass: OdcinekAutobusowy [
    czas [ ^super czas + 2 ]
]
```

`super czas` wywołuje komunikat dla tego samego obiektu, rozpoczynając jego wyszukiwanie w nadklasie klasy definiującej tę metodę. Użycie `self czas` w tym miejscu ponownie uruchomiłoby właśnie definiowaną metodę.

```
| odcinek |
odcinek := OdcinekAutobusowy new ustawCzas: 18.
Transcript show: odcinek czas printString; cr.  "20"
```

Blok i warunek

Blok to fragment kodu w nawiasach kwadratowych. Można go przekazać jako argument. W komunikacie `ifTrue:ifFalse:` odbiorcą jest wartość logiczna. Wykona ona tylko jeden z dwóch bloków.

```
| opis |
opis := (OdcinekAutobusowy new isKindOf: OdcinekPodrozy)
    ifTrue: [ 'odcinek' ]
    ifFalse: [ 'inny obiekt' ].
Transcript show: opis; cr.
```

Wynikiem będzie tekst odcinek. `isKindOf:` uwzględnia podklasy, więc ten sam warunek rozpozna także inne rodzaje odcinków. W Smalltalku mamy predefiniowane bloki różnego typu, np. pętle czy select.

Kolekcja odcinków po kolei

`OrderedCollection` zachowuje kolejność dodawania. Wykonaj ten przykład z klasą ze strony 2. Najpierw przejdź po odcinkach, a następnie wyznacz listę czasów i łączny czas podróży.

```
| etapy czasy razem |
etapy := OrderedCollection new.
etapy add: (OdcinekPodrozy new ustawCzas: 12).
etapy add: (OdcinekPodrozy new ustawCzas: 18).

etapy do: [ :etap |
    Transcript show: etap czas printString; cr ].

czasy := etapy collect: [ :etap | etap czas ].
razem := czasy inject: 0 into: [ :suma :czas |
    suma + czas ].
Transcript show: razem printString; cr.
```

`do:` wykonuje blok dla każdego elementu. Zmienna `:etap` przyjmuje po kolei oba odcinki, więc zobaczysz 12 i 18 w osobnych wierszach.

`collect:` buduje nową kolekcję z wyników bloku. W czasy znajdują się teraz dwie liczby w tej samej kolejności co odcinki.

`inject:into:` przenosi wynik poprzedniego kroku do następnego. Zaczynamy od 0, potem otrzymujemy 0 + 12, a na końcu 12 + 18. Ostatni wiersz wypisze 30.

Mała zmiana do samodzielnej próby

Zastąp drugi odcinek instancją `OdcinekAutobusowy`. Przed uruchomieniem przewidź ostatni wynik. Powinien wzrosnąć do 32, ponieważ `collect:` wysyła czas do każdego rzeczywistego odbiorcy.

Jeden plus dla trzech połączeń, czyli polimorfizm

Poniższe wywołania opisują oczekiwane zachowanie projektu. Przyjmij, że a, b, c i d oznaczają cztery różne odcinki w kolejności podróży.

```
lewa := a + b.              "Trasa: a, b"
dluzsza := lewa + c.        "Trasa: a, b, c"
prawa := c + d.             "Trasa: c, d"
calosc := lewa + prawa.     "Trasa: a, b, c, d"
```

Gdzie zdefiniować plus

W klasie `OdcinekPodrozy` zdefiniuj metodę binarną o nagłówku `+ drugi`. W `Trasa` zdefiniuj osobną metodę o takim samym nagłówku. Smalltalk wybiera metodę według klasy odbiorcy po lewej stronie. Wybrana metoda może następnie sprawdzić obiekt po prawej.

```
(drugi isKindOf: OdcinekPodrozy)
    ifTrue: [ "dolacz jeden odcinek do wyniku" ]
    ifFalse: [ "sprawdz, czy drugi jest trasa" ].
```

To szkic decyzji wewnątrz metody `Trasa`. Dla trasy dołącz kolejno jej odcinki. Jeśli argument nie pasuje do żadnego obsługiwanego przypadku, zgłoś błąd przez `self error: 'Oczekiwano odcinka lub trasy'`. Zakończ udane łączenie, zwracając nową trasę przez `^`.

Jak zachować oryginały

```
wynik := OrderedCollection new.
wynik addAll: leweEtapy.
wynik addAll: praweEtapy.
```

Ten fragment łączy dwie kolekcje odcinków. Użyj wyniku jako zawartości nowej trasy. Dla pojedynczego odcinka użyj `add:`. Gdyby wynik wskazywał na `leweEtapy`, dopisywanie zmieniłoby także lewą trasę. Nowa kolekcja może zawierać te same odcinki, o ile podczas łączenia nie zmieniasz ich stanu.

Mini testy planera

Uruchom te próby po napisaniu własnych metod `+`. W przykładzie zakładamy, że komunikat odcinki zwraca uporządkowaną sekwencję etapów. Jeśli Twój projekt używa innej nazwy, dostosuj wywołania.

```
| a b c d lewa prawa dluzsza calosc sprawdz blad |
sprawdz := [ :warunek :opis |
    warunek ifFalse: [ Error signal: opis ] ].
a := OdcinekPodrozy new ustawCzas: 5.
b := OdcinekPodrozy new ustawCzas: 10.
c := OdcinekPodrozy new ustawCzas: 15.
d := OdcinekPodrozy new ustawCzas: 20.

lewa := a + b.
prawa := c + d.
dluzsza := lewa + c.
calosc := lewa + prawa.

sprawdz value: (lewa isKindOf: Trasa) value: 'Typ wyniku'.
sprawdz value: (dluzsza odcinki asArray = (Array with: a with: b with: c))
    value: 'Trasa + odcinek'.
sprawdz value: (calosc odcinki asArray = (Array with: a with: b with: c with:
d))
    value: 'Trasa + trasa'.
sprawdz value: (lewa odcinki asArray = (Array with: a with: b))
    value: 'Zmieniona lewa trasa'.
sprawdz value: (prawa odcinki asArray = (Array with: c with: d))
    value: 'Zmieniona prawa trasa'.

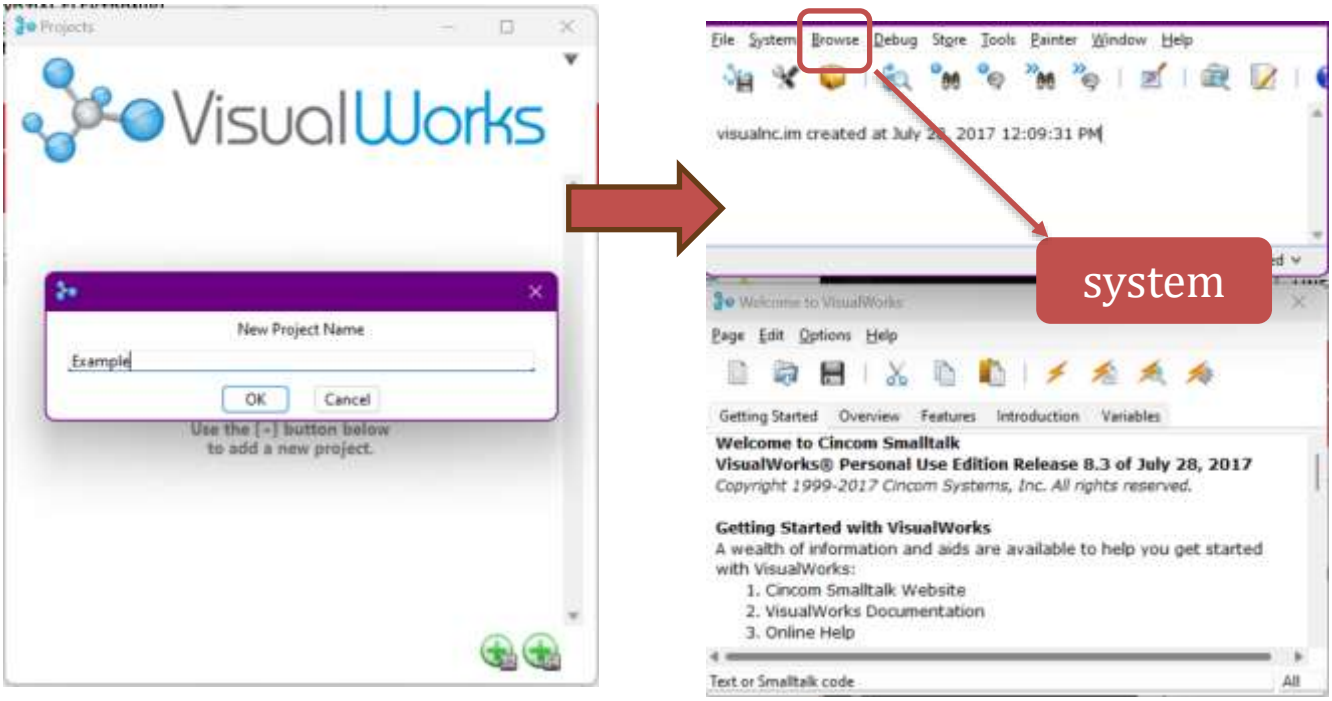
blad := false.
[ lewa + 7 ] on: Error do: [ :e | blad := true ].
sprawdz value: blad value: 'Brak bledu dla liczby'.
Transcript show: 'OK'; cr.
```

Blok `sprawdz` przyjmuje dwa argumenty przez `value:value:`. Gdy warunek jest fałszywy, test zatrzymuje się z opisem próby. Napis OK oznacza, że wszystkie sprawdzenia przeszły. Ostatnia próba wykrywa dowolny Error. Aby sprawdzić przyczynę, obejrzyj także treść wyjątku.

Praca w Pharo

Przykłady definicji w tym skrypcie mają składnię GNU Smalltalk. W Pharo (np. Cincom VisualWorks lub Squeak) klasy i jej zmienne tworzy się w przeglądarce klas, a każdą metodę zatwierdza osobno, bez zewnętrznych nawiasów kwadratowych. Kod jest od razu „wgrywany” w środowisko, więc kontrola błędów następuje w chwili zatwierdzania. W typowej klasie Pharo dziedziczone `new` wywołuje `initialize`.

Tworzenie projektu w VSWork

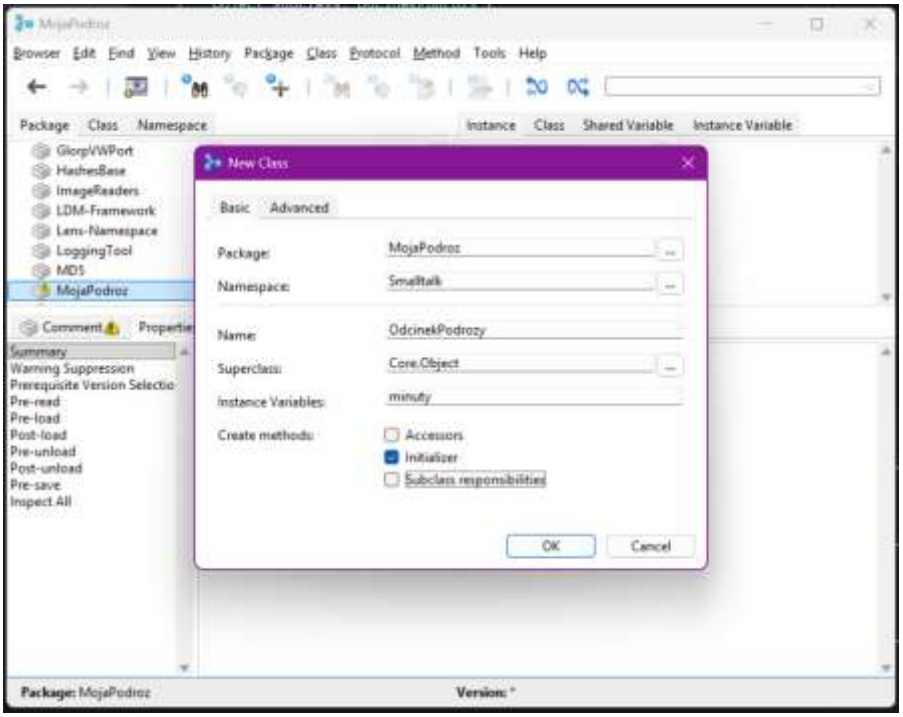


Nowy pakiet

Package -> New Package -> (twoja nazwa)

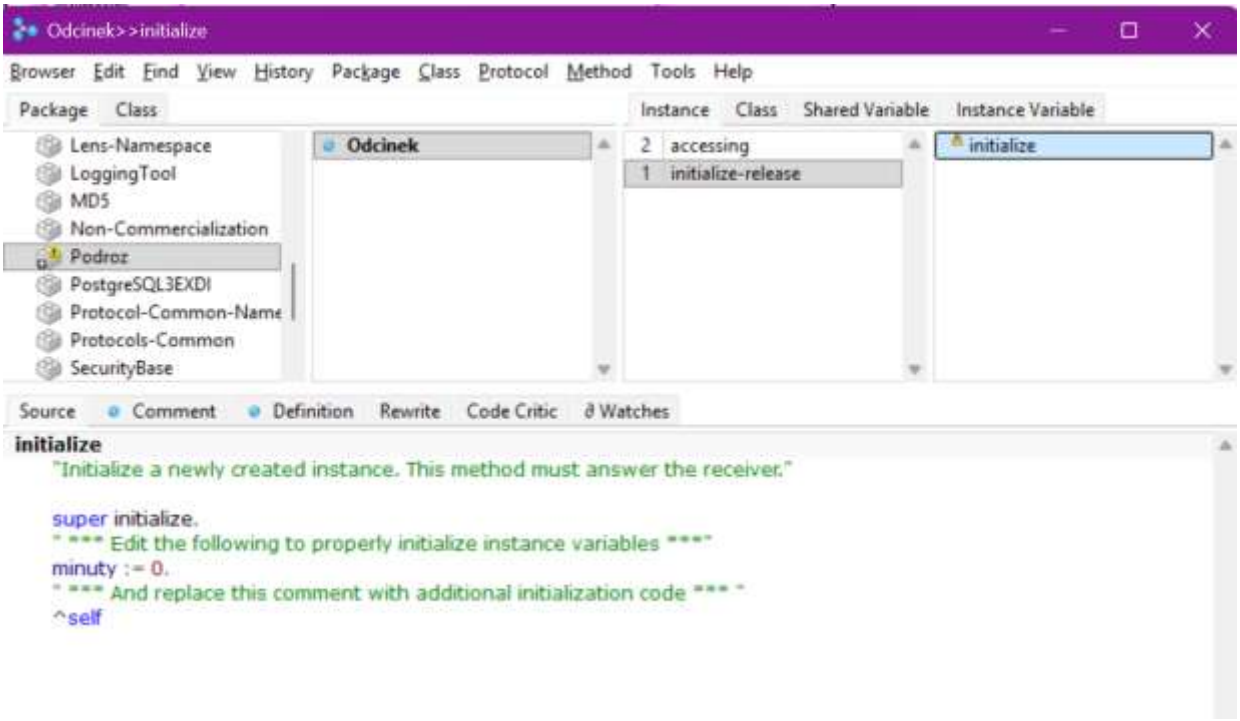
Nowa klasa

Class -> New Class (ustal nazwę klasy, pola, inne parametry, jeśli są potrzebne)

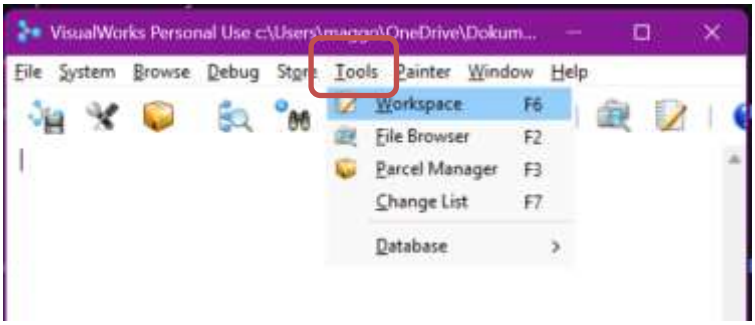


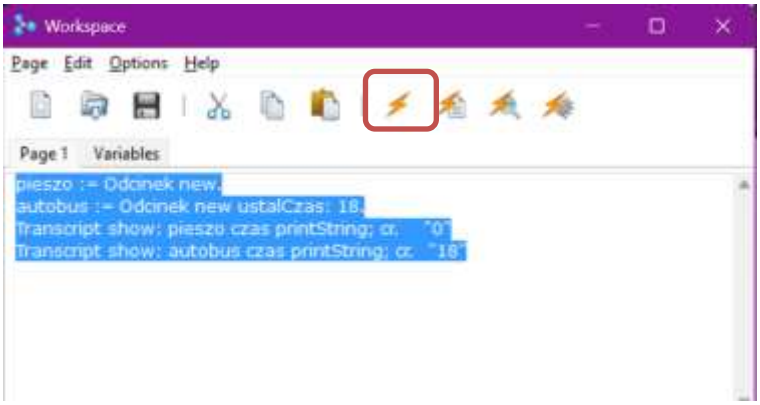
Komunikaty

Po kliknięciu szablonu, wpisać kod, prawoklik -> Accept



Uruchomienie testów





Pomoce

GNU Smalltalk User's Guide (gnu.org/software/smalltalk/manual/gst.pdf).